JAVA CORE CONCEPTS

Loops in Java (Complete Guide)

Iterative structures, for, while, do-while loops, nested loops, and control keywords

> In programming, we often need to repeat a task multiple times. A **loop** is a control structure that executes a block of code repeatedly as long as a specified condition is satisfied. Loops eliminate redundant code, reduce errors, and are fundamental to Data Structures and Algorithms (DSA).

1. Why Do We Need Loops?

Loops allow programs to scale efficiently. Instead of repeating statements manually:

```
// Unscalable manual approach:
System.out.println(1);
System.out.println(2);
System.out.println(3);
System.out.println(4);
System.out.println(5);
```

A loop handles iterations dynamically. Java provides three main loop constructs:

- `for` loop — Best when iteration count is known in advance
- `while` loop — Best when repetition depends on a dynamic condition
- `do-while` loop — Guarantees execution at least once

2. Java for Loop

The `for` loop is used when you know in advance how many times a code block should run.

Syntax:

```
for (initialization; condition; update) {
    // Code block to repeat
}
```

Flow of Execution: `Initialization` (runs once) → `Condition Check` → `Loop Body` → `Update` → `Condition Check` (repeats until false).

Example 1: Print Numbers 1 to 5

```
public class Main {
    public static void main(String[] args) {
        for (int day = 1; day <= 5; day++) {
            System.out.println(day);
        }
    }
}
```

```
Output:
1
```

```
2
3
4
5
```

Example 2: Print Even Numbers in Steps

```java
public class Main {
    public static void main(String[] args) {
        for (int value = 2; value <= 10; value += 2) {
            System.out.println(value);
        }
    }
}
```

3. Java while Loop

The `while` loop keeps running as long as its boolean condition remains `true`. It is ideal when the total iteration count is unknown beforehand.

Example 3: Countdown Using while Loop

```java
public class Main {
    public static void main(String[] args) {
        int modulesLeft = 4;

        while (modulesLeft >= 1) {
            System.out.println("Modules left: " + modulesLeft);
            modulesLeft--;
        }
    }
}
```

```
Output:
Modules left: 4
Modules left: 3
Modules left: 2
Modules left: 1
```

4. Java do-while Loop

Unlike `for` and `while` (entry-controlled loops), `do-while` is an **exit-controlled loop**. It executes the loop body **at least once** before checking the condition.

Syntax:

```java
do {
    // Code block
} while (condition); // Note the trailing semicolon
```

Example 4: do-while Runs At Least Once

```
public class Main {
    public static void main(String[] args) {
        int score = 10;

        do {
            System.out.println("This runs once even if condition is false.");
        } while (score > 50);
    }
}
```

5. Loop Type Comparison

Loop Construct	Condition Check Timing	Minimum Executions	Primary Use Case
for	Entry-controlled (Before body)	0	Fixed iteration count, array indexing, ranges
while	Entry-controlled (Before body)	0	Dynamic conditions, state-dependent repetition
do-while	Exit-controlled (After body)	1	Menu systems, interactive prompts

6. Nested Loops & Pattern Printing

A loop inside another loop is a **nested loop**. The outer loop manages rows/major cycles, while the inner loop completes all its iterations for every single outer iteration.

```
public class Main {
    public static void main(String[] args) {
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 3; col++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

```
Output:
* * *
* * *
* * *
```

7. Loop Control Keywords: break and continue

continue Keyword

Skips the current iteration and jumps to the next update step.

break Keyword

Exits the loop immediately, skipping all remaining iterations.

```java
for (int i = 1; i <= 5; i++) {
    if (i == 4) break;
    System.out.println(i);
} // Output: 1, 2, 3
```

```java
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
} // Output: 1, 2, 4, 5
```

⚠️ Common Beginner Mistakes

- **Missing Update Statement:** Omitting `i++` in a `while` loop leads to an infinite execution loop.
- **Extra Semicolon:** Writing `for (int i=0; i<5; i++);` terminates the loop header immediately, detaching the code block below.
- **Incorrect Loop Boundaries:** Logic errors like `i >= 5` during initialization cause zero iterations.

💡 Interview & DSA Relevance

Loop constructs form the core execution mechanism across data structure traversal and algorithms:

- Array & Matrix traversals (1D, 2D grids)
- Searching (Binary Search, Linear Search) & Sorting (Bubble, Selection, Insertion)
- Sliding Window & Two Pointer techniques
- Dynamic Programming tabular transitions

📝 Practice Questions

1. Print numbers from 10 down to 1 using a `while` loop.
2. Print the multiplication table of 7 using a `for` loop.
3. Calculate the sum of numbers from 1 to `N`.
4. Print a right-angled triangle pattern of size `N` using nested loops.